

1. Explain the following terms:

- (a) (2 points) Processor affinity
- (b) (2 points) Critical section
- (c) (2 points) Safe sequence of processes
- (d) (2 points) The worst-fit strategy of contiguous memory allocation
- (e) (2 points) Inverted page table

2. Scheduling – Consider the following two processes P_1 and P_2 , each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively:

4. 2. 4. 2 *4* *2. 4*

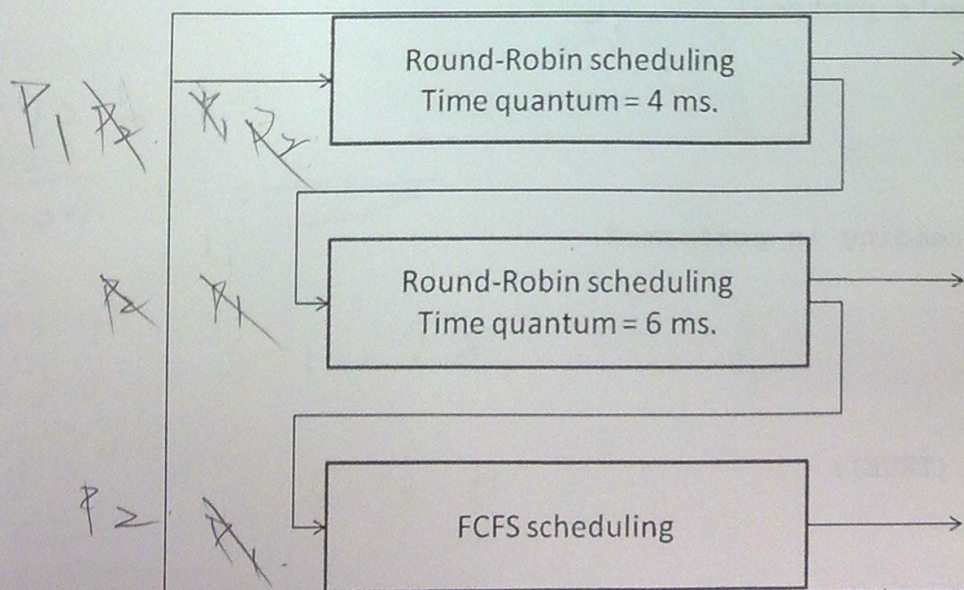
P_1 : CPU burst: 8 milliseconds.	I/O burst: 4 milliseconds.	CPU burst: 8 milliseconds.
------------------------------------	----------------------------	----------------------------

2 *4. 4. 4*

P_2 : CPU burst: 2 milliseconds.	I/O burst: 2 milliseconds.	CPU burst: 16 milliseconds.
------------------------------------	----------------------------	-----------------------------

P_2 : *cpu* *io* *cpu*

- (a) (10 points) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that a **high-level process will preempt a low-level process** and a process will be scheduled to a **lower level** if it is preempted.



- (b) (10 points) What is the average waiting time for the scheduling algorithm?

3. Synchronization

- (a) (10 points) Complete the following entry and exit code fragments of Peterson's solution for process P_0 .
- (b) (10 points) show that the code fragments satisfy the three requirements of critical section solution.

```
// shared data for Peterson's solution
```

```
int boolean flag[2];  
turn = 1;
```

```
do { flag[0] = true;
```

```
while (turn == 1 & flag[1] == true) {
```

```
CRITICAL SECTION
```

```
flag[0] = false;
```

```
REMAINDER SECTION
```

```
} while (TRUE);
```

mutex
progress

4. (10 points) In the first readers-writers problem, readers have higher priorities to access a shared object (i.e., file). Design the entry and exit sections of the reader processes using the following (shared) data structures:

```
semaphore mutex, wrt; // initialized to 1
```

```
int readcount; // initialized to 0
```

```
// reader's protocol
```

```
do {
```

```
// reading is performed
```

```
} while (TRUE);
```


5. Deadlocks:

- (10 points) **Explain** the four necessary conditions of a deadlock.
- (5 points) Can an **UNSAFE** system back to a **SAFE** state? Explain your answer.
- (10 points) Consider the following snapshot of a **SAFE** system; can a request $(0,4,2,0)$ from P_1 be granted? Show your decision by using the banker's algorithm.

	Allocation				Max			
	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2
P_1	1	0	0	0	1	7	5	0
P_2	1	3	5	4	2	3	5	6
P_3	0	6	3	2	0	6	5	2
P_4	0	0	1	4	0	6	5	6

Available			
A	B	C	D
1	5	2	0

mutex progress

Need

P_0	0	0	0	0
P_1	0	7	5	0
P_2	1	0	0	2
P_3	0	0	2	0
P_4	0	6	4	2

6. Paging – Consider a paging system that supports 4-bit logical address and has 64 bytes physical memory. The size of a page is 4 bytes.

- (5 points) If the size of a page table entry is 2 bytes (first byte for frame number & second byte for protection bits), then how many bytes are required for a page table?
- (10 points) The following figure shows the physical memory content of the paging system and the 3rd page table (three-level paging) of a process. Fill the logical content of the process.

1512
0612
1164

1414
1750
2164
1002
1162

1520
12
1532

~~0612~~

1172

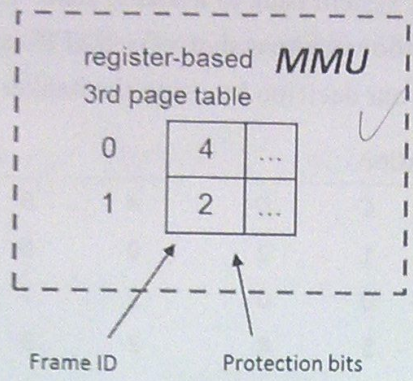
1530

1186

1532
0530

Logical address

content
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31



Physical address

content	content
0	32
1	33
2	34
3	35
4	36
5	37
6	38
7	39
8	40
9	41
10	42
11	43
12	44
13	45
14	46
15	47
16	48
17	49
18	50
19	51
20	52
21	53
22	54
23	55
24	56
25	57
26	58
27	59
28	60
29	61
30	62
31	63